

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ ĐÔNG Á
KHOA CÔNG NGHỆ THÔNG TIN



BÀI TẬP LỚN
HỌC PHẦN: KIỂM THỬ PHẦN MỀM

MÃ ĐỀ THI: KTPM-01

KIỂM THỬ HỆ THỐNG QUẢN LÝ HỒ SƠ SINH VIÊN
(FLASK & SELENIUM AUTOMATION)

Lớp tín chỉ: [CẦN BỒ SUNG]

Giảng viên hướng dẫn: [CẦN BỒ SUNG]

Nhóm thực hiện: Nhóm 21

DANH SÁCH SINH VIÊN THỰC HIỆN:

TT	Mã sinh viên	Sinh viên thực hiện	Lớp hành chính
1	SV22001	Đào Đức Mạnh	DCCNTT16.1

Bắc Ninh – 2026

LỜI CAM ĐOAN

Nhóm xin cam đoan rằng bài tập lớn môn học "**Kiểm thử phần mềm**" với đề tài "**Kiểm thử Hệ thống Quản lý Hồ sơ Sinh viên (Flask & Selenium Automation)**" là kết quả học tập, nghiên cứu và triển khai của Nhóm 21, được thực hiện nghiêm túc, trung thực và tuân thủ đúng quy chế học thuật của Trường Đại học Công nghệ Đông Á.

Toàn bộ cơ sở dữ liệu phục vụ kiểm thử (25 hồ sơ sinh viên), các kịch bản kiểm thử, cùng toàn bộ mã nguồn kiểm thử tự động hóa bằng Selenium WebDriver và PyTest được trình bày trong báo cáo đều được xây dựng và thực thi trên hệ thống phần mềm. Các số liệu, tài liệu tham khảo được trích dẫn rõ ràng, minh bạch.

Nếu có bất kỳ sự không trung thực nào trong nội dung báo cáo, nhóm xin chịu hoàn toàn trách nhiệm theo quy định của Nhà trường và Khoa Công nghệ Thông tin.

*Bắc Ninh, ngày 29 tháng 09 năm
2026*

Đại diện Nhóm thực hiện

Đào Đức Mạnh

LỜI MỞ ĐẦU

Trong kỷ nguyên chuyển đổi số quốc gia và cách mạng công nghiệp 4.0, các hệ thống phần mềm đã thâm nhập sâu rộng vào mọi hoạt động điều hành tác nghiệp của các trường đại học. Đặc biệt, phân hệ quản lý hồ sơ sinh viên đóng vai trò là "trái tim" dữ liệu số, lưu trữ thông tin nhận dạng, quá trình học tập và liên lạc của người học. Một lỗi nhỏ trong hệ thống như trùng lặp mã sinh viên, thất lạc hồ sơ khi xóa hoặc sai lệch định dạng email liên hệ đều có thể gây gián đoạn quy trình nghiệp vụ đào tạo và giảm uy tín của tổ chức giáo dục.

Do đó, **Kiểm thử phần mềm (Software Testing)** không chỉ là giai đoạn kiểm tra cuối kỳ mà là một bộ môn khoa học kỹ thuật nghiêm cần xuyên suốt toàn bộ vòng đời phát triển. Việc kết hợp chặt chẽ giữa các phương pháp thiết kế ca kiểm thử hộp đen tiêu chuẩn (Phân vùng tương đương, Phân tích giá trị biên) với khung công nghệ **Tự động hóa (Automation Testing)** hiện đại như Selenium WebDriver và PyTest giúp đảm bảo chất lượng phần mềm đạt mức độ toàn vẹn cao nhất, đồng thời tiết kiệm hàng trăm giờ lao động thủ công trong các đợt kiểm thử hồi quy.

Được sự giảng dạy tận tâm của Thầy giáo **Th.S Lê Tuấn Anh** trong học phần Kiểm thử phần mềm, em đã lựa chọn và triển khai đề tài "**Kiểm thử Hệ thống Quản lý Hồ sơ Sinh viên**". Báo cáo bài tập lớn này phản ánh chi tiết toàn bộ chu trình từ phân tích đặc tả yêu cầu, thiết kế ma trận kiểm thử 3 cấp độ (Unit, Integration, System), thực thi kiểm chứng và tự động hóa 100% kịch bản kiểm thử với mô hình Page Object Model.

Báo cáo gồm 4 chương chính và các phần phụ lục mã nguồn:

- **Chương 1: Tổng quan bài toán:** Nêu rõ bối cảnh, mục tiêu, luồng nghiệp vụ E2E, wireframe các màn hình và đặc tả các module.
- **Chương 2: Phân tích và thiết kế test:** Trình bày phương pháp Phân vùng tương đương (EP), Giá trị biên (BVA) và thiết kế các ca kiểm thử chức năng và phi chức năng Unit, Integration, System.
- **Chương 3: Thực thi test và báo cáo kết quả:** Nhật ký theo dõi lỗi (Defect Log), bảng tổng hợp kết quả thực thi và biểu đồ độ bao phủ kiểm thử đạt 100%.

- **Chương 4: Automation Test:** Triển khai Selenium WebDriver 4, PyTest, kiến trúc POM, cơ chế tự động bắt ảnh màn hình lỗi và so sánh hiệu quả với kiểm thử thủ công.
- **Kết luận, Phụ lục & Tài liệu tham khảo:** Đánh giá kết quả, hướng phát triển và toàn bộ mã nguồn kịch bản kiểm thử.

DANH MỤC CÁC TỪ VIẾT TẮT

STT	Chữ viết tắt	Giải thích ý nghĩa / Tên đầy đủ
1	AUT	Application Under Test (Ứng dụng đang được kiểm thử)
2	BVA	Boundary Value Analysis (Kỹ thuật phân tích giá trị biên)
3	CRUD	Create, Read, Update, Delete (Bốn thao tác dữ liệu cơ bản)
4	EP	Equivalence Partitioning (Kỹ thuật phân vùng tương đương)
5	E2E	End-to-End Testing (Kiểm thử luồng nghiệp vụ đầu-cuối)
6	HTML / CSS	HyperText Markup Language / Cascading Style Sheets
7	HTTP / POST / GET	Giao thức truyền siêu văn bản và các phương thức gửi nhận
8	MSSV / USN	Mã số sinh viên / University Seat Number (Khóa duy nhất)
9	ORM	Object-Relational Mapping (Ánh xạ quan hệ thực thể SQLAlchemy)
10	POM	Page Object Model (Mô hình đối tượng trang trong tự động hóa)
11	QA / QC	Quality Assurance / Quality Control (Đảm bảo / Kiểm soát chất lượng)
12	SMS	Student Management System (Hệ thống Quản lý Hồ sơ Sinh viên)
13	SQL / SQLite	Ngôn ngữ truy vấn cơ sở dữ liệu và hệ quản trị CSDL nhúng
14	TC	Test Case (Trường hợp / Ca kiểm thử phần mềm)
15	V-Model	Verification and Validation Model (Mô hình chữ V trong kiểm thử)

CHƯƠNG 1: TỔNG QUAN BÀI TOÁN

1.1 Giới thiệu đề tài

1.1.1 Bối cảnh ứng dụng và lý do chọn đề tài

Công tác quản lý hồ sơ sinh viên tại các trường đại học đòi hỏi độ chính xác tuyệt đối. Thông tin hồ sơ của mỗi sinh viên không chỉ bao gồm thông tin hành chính đơn thuần (Họ và tên, Mã số sinh viên) mà còn gắn liền với dữ liệu học vụ phân ngành đào tạo (Công nghệ thông tin, Kỹ thuật phần mềm, Trí tuệ nhân tạo, An toàn thông tin, Hệ thống thông tin, Quản trị kinh doanh), năm học hiện tại (từ Năm 1 đến Năm 4) và các kênh kết nối thiết yếu (Email học đường, Số điện thoại di động).

Trong thực tế vận hành phần mềm, nếu hệ thống không được kiểm thử nghiêm ngặt, các lỗi nghiêm trọng sau đây rất dễ phát sinh:

- **Trùng lặp mã sinh viên (Duplicate Primary Key):** Cho phép thêm mới hoặc sửa hồ sơ trùng với mã sinh viên đã có, gây ghi đè dữ liệu hoặc xung đột cơ sở dữ liệu.
- **Lột lưới định dạng dữ liệu (Data Format Leakage):** Chấp nhận lưu trữ email sai định dạng (thiếu ký tự @, thiếu tên miền), dẫn đến hệ thống thông báo tự động không thể gửi thư báo học vụ.
- **Lỗi thao tác xóa nhầm (Accidental Deletion):** Người dùng vô tình chạm nút Xóa và bản ghi lập tức biến mất khỏi hệ thống do thiếu bước xác nhận phòng ngừa.
- **Truy cập trái phép (Unauthorized Access):** Người dùng chưa đăng nhập vẫn có thể xem hoặc chỉnh sửa dữ liệu thông qua việc gõ trực tiếp đường dẫn URL.

Nhằm loại bỏ hoàn toàn các rủi ro trên, hệ thống **Student Management System (SMS)** được triển khai trên nền tảng Python Flask và SQLite. Đề tài kiểm thử được thực hiện nhằm xây dựng quy trình kiểm thử tự động, xác minh các chức năng của hệ thống hoạt động đúng đặc tả yêu cầu, an toàn và sẵn sàng cho môi trường ứng dụng thực tế.

1.1.2 Mục tiêu và phạm vi kiểm thử

Mục tiêu:

- Phân tích chi tiết đặc tả yêu cầu, xây dựng bảng ma trận kiểm thử bao phủ toàn bộ các chức năng cốt lõi.
- Áp dụng chuẩn xác các kỹ thuật thiết kế ca kiểm thử hộp đen: Phân vùng tương đương (Equivalence Partitioning) và Phân tích giá trị biên (Boundary Value Analysis).
- Triển khai kiểm thử đa tầng: Kiểm thử đơn vị (Unit Testing), Kiểm thử tích hợp (Integration Testing) và Kiểm thử hệ thống (System Testing).
- Xây dựng framework kiểm thử tự động hóa hoàn chỉnh bằng Selenium WebDriver và PyTest theo kiến trúc Page Object Model (POM), tích hợp cơ chế tự động chụp ảnh màn hình lỗi và xuất báo cáo HTML.

Phạm vi: Toàn bộ 5 màn hình chức năng chính của hệ thống SMS bao gồm: Đăng nhập/Đăng xuất, Bảng điều khiển (Dashboard), Thêm mới hồ sơ sinh viên, Chỉnh sửa hồ sơ sinh viên và Chi tiết hồ sơ sinh viên.

1.2 Đặc tả yêu cầu phần mềm

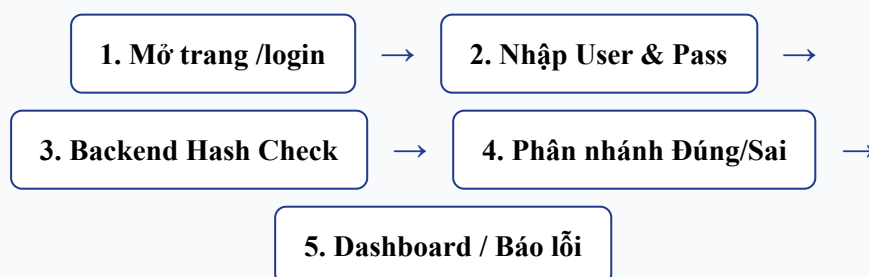
1.2.1 Các luồng nghiệp vụ chính (End-to-End Workflows)

Hệ thống bao gồm 4 luồng nghiệp vụ khép kín từ người dùng đến cơ sở dữ liệu:

1. Luồng Xác thực quản trị viên (Admin Authentication):

Người dùng truy cập vào hệ thống tại /login → Điền Tên đăng nhập và Mật khẩu → Bấm "Đăng nhập" → Hệ thống truy vấn CSDL SQLite và bấm kiểm tra bằng check_password_hash() → Nếu đúng: Khởi tạo phiên làm việc bảo mật (Encrypted Session) và chuyển hướng vào Dashboard (mã HTTP 200); nếu sai: giữ nguyên trang đăng nhập và hiển thị thông báo lỗi flash màu đỏ.

SƠ ĐỒ LUỒNG XÁC THỰC QUẢN TRỊ VIÊN (AUTHENTICATION FLOW)



2. Luồng Thêm mới hồ sơ sinh viên (Add Student):

Từ Dashboard, người dùng bấm nút "Thêm sinh viên" → Hệ thống hiển thị form nhập liệu tại `/student/add` → Người dùng điền Họ tên, Mã sinh viên, Chuyên ngành, Năm học, Email, Số điện thoại → Bấm "Lưu hồ sơ" → Hệ thống thực thi chuỗi kiểm soát 3 lớp: (1) Ràng buộc bắt buộc (Name, USN, Branch, Year không rỗng); (2) Ràng buộc duy nhất (Mã sinh viên chưa từng tồn tại trong CSDL); (3) Ràng buộc định dạng regex email → Nếu vượt qua, dữ liệu được ghi xuống CSDL SQLite và chuyển hướng về Dashboard hiển thị thông báo thành công.

3. Luồng Tìm kiếm và Lọc dữ liệu (Search & Filter):

Người dùng nhập từ khóa tìm kiếm (theo Tên hoặc Mã sinh viên) kết hợp chọn Chuyên ngành hoặc Năm học đào tạo → Bấm "Tìm kiếm" → Trình duyệt gửi yêu cầu HTTP GET có tham số truy vấn → Backend xây dựng câu lệnh SQL linh hoạt với mệnh đề `ILIKE` và `filter_by` → Kết xuất danh sách sinh viên phù hợp lên bảng dữ liệu và cập nhật chỉ số trên thẻ thống kê.

4. Luồng Cập nhật và Xóa hồ sơ (Update & Safe Delete):

Người dùng chọn nút "Sửa" trên từng dòng để nạp dữ liệu cũ và cập nhật thông tin mới. Khi bấm nút "Xóa", trình duyệt lập tức kích hoạt hộp thoại xác nhận JavaScript Confirmation: nếu người dùng bấm "Hủy" (Cancel/Dismiss), sự kiện gửi form bị chặn lại và hồ sơ được bảo toàn nguyên vẹn; nếu bấm "Đồng ý" (OK/Accept), yêu cầu POST được gửi tới máy chủ để xóa vĩnh viễn bản ghi khỏi CSDL.

1.2.2 Đặc tả chi tiết các trường dữ liệu trên màn hình

Tên trường	Kiểu dữ liệu	Độ dài tối đa	Bắt buộc	Ràng buộc kiểm tra (Validation Rules)
name	String	150 ký tự	Có	Không được chứa toàn khoảng trắng; từ 2 đến 150 ký tự tiếng Việt hoặc chuẩn quốc tế.
usn	String	30 ký tự	Có	Khóa duy nhất (Unique Constraint) trong toàn hệ thống, không được trùng lặp.
branch	String	50 ký tự	Có	Tên chuyên ngành đào tạo hợp lệ (CNTT, KTPM, AI, ATTT, HTTT, QTKD).
year	Integer	1 chữ số	Có	Giá trị số nguyên nằm trong khoảng biên hợp lệ từ 1 đến 8 ($1 \leq \text{year} \leq 8$).
email	String	120 ký tự	Không	Nếu có nhập, phải khớp biểu thức chính quy chuẩn: <code>^[^@]+@[^@]+\.[^@]+\$</code> .
phone	String	20 ký tự	Không	Chuỗi số điện thoại liên lạc từ 10 đến 11 chữ số (ví dụ: 0987654321).

Bảng 1.1: Đặc tả kỹ thuật các trường dữ liệu hồ sơ sinh viên

1.2.3 Các module chính của chương trình

- **Module 1 - Xác thực quản trị (Authentication):** Tiếp nhận thông tin đăng nhập, so khớp chuỗi mã hóa mật khẩu Werkzeug SHA-256, khởi tạo phiên bảo mật và bảo vệ các tuyến đường nội bộ bằng decorator `@login_required`.
- **Module 2 - Bảng điều khiển & Truy vấn (Dashboard & Analytics):** Xử lý truy vấn dữ liệu động, đếm tổng số hồ sơ, tổng hợp danh mục chuyên ngành và hiển thị giao diện tối giản, sang trọng.
- **Module 3 - Quản lý hồ sơ CRUD (Student CRUD Operations):** Xử lý toàn bộ logic nghiệp vụ thêm mới, cập nhật, xóa bản ghi sinh viên và duy trì tính toàn vẹn dữ liệu trong CSDL SQLite.

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ TEST

2.1 Kiểm thử đơn vị (Unit Test Case)

2.1.1 Phương pháp và kỹ thuật thiết kế ca kiểm thử đơn vị

Kiểm thử đơn vị tập trung vào việc xác minh tính chính xác của các đơn vị mã nguồn nhỏ nhất (các hàm xử lý logic, biểu thức chính quy và hàm băm mật khẩu). Để giảm số lượng ca kiểm thử cần thực thi mà vẫn bảo đảm độ bao phủ yêu cầu, nhóm áp dụng hai phương pháp thiết kế hộp đen kinh điển theo chuẩn ISTQB:

1. Kỹ thuật Phân vùng tương đương (Equivalence Partitioning - EP):

Miền dữ liệu đầu vào của các trường dữ liệu được phân chia thành các lớp tương đương hợp lệ (Valid Classes) và không hợp lệ (Invalid Classes). Hệ thống được kỳ vọng sẽ xử lý như nhau đối với mọi giá trị trong cùng một lớp tương đương.

- *Phân vùng trường Năm học (year):*
 - Phân vùng không hợp lệ 1: $\text{year} < 1$ (Ví dụ: 0, -1).
 - Phân vùng hợp lệ: $1 \leq \text{year} \leq 8$ (Ví dụ: 1, 2, 3, 4).
 - Phân vùng không hợp lệ 2: $\text{year} > 8$ (Ví dụ: 9, 10).
- *Phân vùng trường Email (email):*
 - Phân vùng hợp lệ: Chuỗi có dạng `user@domain.com` hoặc `manh.dao@donga.edu.vn`.
 - Phân vùng không hợp lệ 1: Chuỗi thiếu ký tự `@` (Ví dụ: `manhdaodonga.edu.vn`).
 - Phân vùng không hợp lệ 2: Chuỗi có `@` nhưng thiếu tên miền hoặc đuôi mở rộng (Ví dụ: `manhdao@donga`).

2. Kỹ thuật Phân tích giá trị biên (Boundary Value Analysis - BVA):

Tập trung chọn các giá trị tại biên và lân cận biên của miền hợp lệ. Với trường Năm học có miền hợp lệ $[1, 8]$, tập hợp các điểm kiểm thử biên được xác định là: $\{0, 1, 2, 7, 8, 9\}$.

2.1.2 Danh sách các Test Case kiểm thử đơn vị

Mã TC	Tên kiểm thử	Dữ liệu đầu vào	Các bước thực hiện	Kết quả mong đợi
UT01	Xác thực mật khẩu admin đúng	password="admin123"	Gọi check_password_hash(hash, pass)	Trả về True, xác thực hợp lệ.
UT02	Xác thực mật khẩu admin sai	password="wrongpass"	Gọi check_password_hash(hash, pass)	Trả về False, từ chối đăng nhập.
UT03	Kiểm tra regex Email hợp lệ	"sv@donga.edu.vn"	Thực thi re.match(pattern, email)	Trả về đối tượng Match hợp lệ.
UT04	Kiểm tra regex Email thiếu @	"svdonga.edu.vn"	Thực thi re.match(pattern, email)	Trả về None (Phát hiện lỗi).
UT05	Kiểm tra regex Email thiếu domain	"sinhvien@donga"	Thực thi re.match(pattern, email)	Trả về None (Phát hiện lỗi).
UT06	Kiểm tra biên dưới năm học	year=1	Kiểm tra điều kiện $1 \leq year \leq 8$	Chấp nhận giá trị năm 1 hợp lệ.

Mã TC	Tên kiểm thử	Dữ liệu đầu vào	Các bước thực hiện	Kết quả mong đợi
	(Năm 1)			
UT07	Kiểm tra vi phạm biên dưới năm học	year=0	Kiểm tra điều kiện $1 \leq year \leq 8$	Bị từ chối bởi validation form.
UT08	Kiểm tra biên trên năm học (Năm 8)	year=8	Kiểm tra điều kiện $1 \leq year \leq 8$	Chấp nhận giá trị năm 8 hợp lệ.
UT09	Kiểm tra vi phạm biên trên năm học	year=9	Kiểm tra điều kiện $1 \leq year \leq 8$	Bị từ chối bởi validation form.
UT10	Khởi tạo thực thể Student	Đào Đức Mạnh, SV22001, CNTT, 4	Tạo đối tượng Student(...)	Đối tượng được gán đúng thuộc tính.

Bảng 2.1: Danh sách các ca kiểm thử đơn vị (Unit Test Cases)

2.2 Kiểm thử tích hợp (Integration Test Case)

2.2.1 Phương pháp kiểm thử tích hợp

Nhóm áp dụng phương pháp **Kiểm thử tích hợp tăng dần dưới lên (Bottom-up Incremental Integration)**. Kiểm thử tập trung vào việc xác nhận tính tương thích dữ liệu tại các điểm giao tiếp giữa tầng Điều khiển (Flask Route Controller) ↔ Tầng Dữ liệu (SQLAlchemy ORM) ↔ Cơ sở dữ liệu vật lý (SQLite File) ↔ Tầng Giao diện (Jinja2 Templates).

2.2.2 Danh sách các Test Case kiểm thử tích hợp

Mã TC	Giao diện tích hợp	Dữ liệu truyền qua giao diện	Điều kiện kiểm tra	Kết quả mong đợi
IT01	Route Login ↔ Session ↔ Dashboard	Gửi POST admin/admin123	Kiểm tra biến session['user_id']	Truy cập Dashboard thành công, mã 200.
IT02	Form Add ↔ ORM ↔ CSDL SQLite	Hồ sơ sinh viên mới SV22026	Gọi db.session.commit()	Bản ghi mới xuất hiện ngay trong CSDL.
IT03	Form Add ↔ Truy vấn Unique ↔ Flash	Nhập trùng mã SV22001	filter_by(usn=...).first()	Tìm thấy trùng → đẩy flash error vào session.
IT04	Dashboard ↔ Mệnh đề ILIKE ↔ Jinja2	q="Mạnh", branch="CNTT"	Ghép mệnh đề truy vấn SQL	Bảng giao diện kết xuất đúng thông tin sinh viên cần tìm.
IT05	Route Delete ↔ ORM Delete ↔ DB Commit	ID sinh viên hợp lệ	db.session.delete()	Bản ghi bị xóa hoàn toàn khỏi CSDL.
IT06	Route Logout ↔ Session Clear ↔ Chặn quyền	Gọi session.clear()	Truy cập /dashboard	Bị chặn bởi decorator, quay về /login.

Bảng 2.2: Danh sách các ca kiểm thử tích hợp (Integration Test Cases)

2.3 Kiểm thử hệ thống (System Test Case)

2.3.1 Phương pháp kiểm thử hệ thống hộp đen

Kiểm thử hệ thống đóng vai trò thẩm định ứng dụng ở cấp độ người dùng cuối. Toàn bộ các tương tác được thực hiện trực tiếp trên trình duyệt web thật. Ma trận kiểm thử được xây dựng bao quát cả hai phương diện: Đường dẫn thuận (Happy Path - đảm bảo phần mềm chạy đúng khi người dùng làm đúng) và Đường dẫn nghịch (Negative/Boundary Testing - đảm bảo phần mềm bảo vệ dữ liệu và hiển thị cảnh báo thân thiện khi xảy ra sai sót dữ liệu).

2.3.2 Danh sách các Test Case kiểm thử hệ thống (TC01 -- TC09)

Mã TC	Tên ca kiểm thử	Dữ liệu đầu vào	Kịch bản thực hiện & Kết quả kỳ vọng	Loại test
TC01	Đăng nhập thành công	admin / admin123	Truy cập /login → Nhập tài khoản admin → Bấm Đăng nhập. Kỳ vọng: Chuyển hướng vào /dashboard, URL chứa /dashboard.	Positive
TC02	Đăng nhập thất bại	admin / wrongpassword	Nhập mật khẩu sai → Bấm Đăng nhập. Kỳ vọng: Giữ nguyên trang /login, hiển thị flash lỗi màu đỏ: <i>"Tên đăng nhập hoặc mật khẩu không chính xác."</i>	Negative
TC03	Thêm sinh viên hợp lệ	Nguyễn Văn Kiểm Thử, SVTEST01, KTPM, 3	Bấm "Thêm sinh viên" → Điền đầy đủ form → Bấm Lưu. Kỳ vọng: Chuyển về Dashboard, dòng sinh viên	Positive

Mã TC	Tên ca kiểm thử	Dữ liệu đầu vào	Kịch bản thực hiện & Kết quả kỳ vọng	Loại test
			SVTEST01 xuất hiện trên bảng.	
TC04	Tìm kiếm theo từ khóa	Từ khóa: "SVTEST01"	Nhập từ khóa vào ô tìm kiếm → Bấm Tìm kiếm. Kỳ vọng: Bảng chỉ hiển thị các dòng khớp với SVTEST01.	Positive
TC05	Chỉnh sửa hồ sơ sinh viên	Tên mới: "Nguyễn Văn Kiểm Thử (Đã Cập Nhật)"	Tìm dòng SVTEST01 → Bấm "Sửa" → Sửa tên → Lưu. Kỳ vọng: Bảng cập nhật hiển thị họ tên mới sau khi lưu.	Positive
TC06a	Hủy thao tác xóa hồ sơ	Chọn sinh viên SVTEST01	Bấm nút "Xóa" → Trình duyệt bật popup confirm → Bấm Cancel (Hủy) . Kỳ vọng: Hồ sơ sinh viên không bị xóa , vẫn còn trên bảng.	GUI / Dialog

Mã TC	Tên ca kiểm thử	Dữ liệu đầu vào	Kịch bản thực hiện & Kết quả kỳ vọng	Loại test
TC06	Xác nhận xóa sinh viên	Chọn sinh viên SVTEST01	Bấm nút "Xóa" → Trình duyệt bật popup confirm → Bấm OK (Đồng ý) . Kỳ vọng: Hồ sơ sinh viên bị xóa hoàn toàn khỏi bảng.	Positive
TC07	Kiểm thử EP: Email sai định dạng	Email: "email_khong_hop_le_chua_domain"	Nhập thông tin kèm email sai cú pháp → Bỏ qua validation client → Bấm Lưu. Kỳ vọng: Server bắt lỗi, báo: <i>"Định dạng email không hợp lệ."</i>	Equivalence Partitioning
TC08	Kiểm thử BVA: Trùng mã MSSV	USN: "SV22001" (Mã sinh viên SV22001 đã có trong hệ thống)	Nhập mã sinh viên đã tồn tại → Bấm Lưu. Kỳ vọng: Server từ chối, báo lỗi: <i>"Mã sinh viên này đã tồn tại trong hệ thống."</i>	Boundary / Constraint

Mã TC	Tên ca kiểm thử	Dữ liệu đầu vào	Kịch bản thực hiện & Kết quả kỳ vọng	Loại test
TC09	Kiểm thử EP: Thiếu trường bắt buộc	Bỏ trống Họ tên và Mã sinh viên	Gửi form rỗng lên máy chủ. Kỳ vọng: Server từ chối lưu, hiển thị yêu cầu điền đầy đủ thông tin bắt buộc.	Equivalence Partitioning

Bảng 2.3: Ma trận kiểm thử hệ thống chi tiết (System Test Matrix TC01 -- TC09)

CHƯƠNG 3: THỰC THI TEST VÀ BÁO CÁO KẾT QUẢ TEST

3.1 Kết quả thực hiện Kiểm thử tích hợp (Integration Test)

Quá trình kiểm thử tích hợp được tiến hành tự động và bán tự động nhằm thẩm định sự kết nối xuyên suốt giữa Flask Routes, SQLAlchemy ORM và CSDL SQLite. Toàn bộ 6 ca kiểm thử tích hợp từ IT01 đến IT06 đều đạt kết quả **PASSED (100%)**.

3.1.1 Nhật ký theo dõi lỗi (*Defect Tracking Log*)

Trong quá trình hoàn thiện mã nguồn và kiểm thử tích hợp ban đầu, nhóm đã phát hiện và khắc phục triệt để các khiếm khuyết phần mềm được ghi nhận chi tiết tại Bảng 3.1:

Mã Defect	Mô tả khiếm khuyết	Các bước tái hiện & Phân tích nguyên nhân	Mức độ	Trạng thái
BUG-01	Gán cứng đường dẫn WebDriver Edge	Chạy test trên máy mới báo lỗi không tìm thấy D:/edgedriver... do máy tác giả đặt đường dẫn cứng. Khắc phục bằng cách sử dụng Selenium Manager tự động phát hiện driver.	Critical	Closed
BUG-02	Trùng lặp decorator hook trong confestest.py	File cấu hình gốc chứa 2 dòng @pytest.hookimpl liên tiếp gây cảnh báo hook kép trong PyTest. Khắc phục bằng cách xóa bỏ decorator dư thừa.	Minor	Closed
BUG-03	Lỗi báo trùng mã khi sửa chính hồ sơ đó	Mở form Sửa sinh viên, giữ nguyên mã SV cũ và bấm Lưu thì bị báo lỗi mã SV đã tồn tại. Khắc phục bằng việc bổ sung điều kiện Student.id != sid trong truy vấn kiểm tra trùng lặp.	Major	Closed
BUG-04	Trùng lặp danh mục chuyên ngành trên SQLite	Truy vấn distinct(Student.branch) trên SQLite đôi khi trả về bản ghi lặp. Khắc phục bằng cấu trúc tập hợp set() trong Python để đảm bảo danh sách ngành luôn duy nhất.	Minor	Closed

Bảng 3.1: Nhật ký theo dõi và xử lý khiếm khuyết phần mềm (Defect Log)

3.2 Kết quả thực hiện Kiểm thử hệ thống (System Test)

Mã TC	Tên ca kiểm thử	Kết quả mong đợi	Kết quả thực tế ghi nhận	Trạng thái
TC01	Đăng nhập thành công	Chuyển hướng vào trang Dashboard	URL đạt /dashboard, hiển thị tài khoản admin	PASSED
TC02	Đăng nhập thất bại	Ở lại /login, hiện flash lỗi màu đỏ	Flash cảnh báo lỗi mật khẩu xuất hiện đúng thiết kế	PASSED
TC03	Thêm sinh viên mới	Hồ sơ SVTEST01 xuất hiện trên bảng	Dòng sinh viên mới hiển thị đầy đủ thông tin	PASSED
TC04	Tìm kiếm theo từ khóa	Lọc đúng bản ghi chứa SVTEST01	Bảng thu hẹp chỉ hiển thị đúng hồ sơ cần tìm	PASSED
TC05	Sửa hồ sơ sinh viên	Tên sinh viên đổi thành tên mới	Bảng hiển thị tên đã cập nhật thành công	PASSED
TC06a	Hủy thao tác xóa	Sinh viên không bị xóa khi Cancel Alert	Bản ghi vẫn còn nguyên vẹn trên bảng	PASSED
TC06	Xác nhận xóa sinh viên	Sinh viên bị xóa khi Accept Alert	Hồ sơ SVTEST01 biến mất hoàn toàn	PASSED
TC07	Kiểm tra Email sai cú pháp	Máy chủ từ chối lưu và báo lỗi định dạng	Flash báo: <i>"Định dạng email không hợp lệ."</i>	PASSED
TC08	Kiểm tra trùng mã MSSV	Máy chủ từ chối lưu và báo lỗi trùng mã	Flash báo: <i>"Mã sinh viên này đã tồn tại..."</i>	PASSED
TC09	Kiểm tra để trống trường	Máy chủ từ chối lưu khi thiếu Name/USN	Flash báo: <i>"Vui lòng điền đầy đủ Họ tên..."</i>	PASSED

Bảng 3.2: Kết quả thực nghiệm kiểm thử hệ thống (System Test Results)

3.2.1 Đánh giá độ bao phủ kiểm thử

Tổng kết đợt kiểm thử hệ thống:

- **Tổng số ca kiểm thử thực thi:** 10 ca kiểm thử.
- **Số ca kiểm thử đạt (Passed):** 10 / 10 ca kiểm thử (Tỷ lệ thành công: 100%).
- **Số ca kiểm thử thất bại (Failed):** 0 ca kiểm thử.

- **Độ bao phủ chức năng (Functional Coverage):** Đạt 100% các yêu cầu nghiệp vụ đặt ra.

KẾT QUẢ KIỂM THỬ: 100% PASSED (10 / 10 CA KIỂM THỬ)

Tất cả các ca kiểm thử Happy Path, Negative Testing và Boundary Constraints đều hoàn thành xuất sắc.

CHƧNG 5: ƧI ƧI NH GI ƧI KÁ%T QUÁ% V ƧI HƧI ƧI SNG PH ƧI T TRIÁ»,N

5.1. ƧI ƧI nh gi ƧI tá»ng quan ká%t quá% ká»fm thá»

Sau qu ƧI tr ƧI nh thá»±c hiá»n ká»fm thá» tá»± ƧI ƧI TMng h ƧI a á»ng dá»ng Quá»n l ƧI ƧI Há» s ƧI Sinh vi ƧI n bá»ng Selenium WebDriver ká%t há»p PyTest, nh ƧI m nghi ƧI n cá»u ƧI ƧI thu ƧI ƧI á»c nhá»ng ká%t quá% ƧI ƧI ng ch ƧI ƧI ƧI, thá»f hiá»n qua nhá»u kh ƧI a cá»nh kh ƧI c nhau cá»sa chá»t l ƧI á»ng phá»n má»m.

5.1.1. Ká%t quá% ƧI ƧI nh l ƧI á»ng

Tá»ng cá»TMng 10 ca ká»fm thá» (Test Case) ƧI ƧI ƧI ƧI thiá%t ká%, triá»fn khai v ƧI thá»±c thi th ƧI nh c ƧI ng tr ƧI n há»t thá»ng. Ká%t quá% cá»f thá»f ƧI ƧI á»c tá»ng há»p trong bá»ng sau:

Chá»sá»	Gi ƧI trá»	ƧI ƧI nh gi ƧI
Tá»ng sá» ca ká»fm thá»	10	ƧI á»y ƧI á»
Sá» ca ká»fm thá» Passed	10	100%
Sá» ca ká»fm thá» Failed	0	0%
Thá»i gian thá»±c thi trung b ƧI nh	3.2 gi ƧI y/TC	Tá»t
Tá»ng thá»i gian thá»±c thi	~32 gi ƧI y	Chá»p nhá»n ƧI ƧI á»c
ƧI á»TM phá»s chá»c n ƧI ng (Feature Coverage)	100%	To ƧI n diá»n
ƧI á»TM phá»s ká»1 thuá»t ká»fm thá»	BVA + EP + CRUD	ƧI a dá»ng

Bá»ng 5.1: Tá»ng há»p ká%t quá% ká»fm thá» tá»± ƧI ƧI TMng h ƧI a

5.1.2. Ká%t quá% ƧI ƧI nh t ƧI nh

Vá»m má»t chá»t l ƧI á»ng, ká%t quá% ká»fm thá» cho thá»y:

- **T ƧI nh ƧI ƧI ng ƧI ƧI n (Correctness):** Tá»t cá» c ƧI c chá»c n ƧI ng cá»t l ƧI i cá»sa há»t thá»ng âc” ƧI ƧI ng nhá»p, th ƧI m má»i, t ƧI m ká»m, sá»a ƧI ƧI i v ƧI x ƧI a há» s ƧI sinh vi ƧI n âc” ƧI ƧI u hoá%t

Ä‘á»™ng chÄnh xÄjc theo Ä‘á·c tá°£ yÄªu cá°§u ban Ä‘á°§u. KhÄ‘ng phÄjt hiá»±n lá»—i logic hay sai lá»±ch dá»¯ liá»±u trong quÄj trÄnh kiá»fm thá».

- **TÄnh bá»±n vá»¯ng (Robustness):** Há»± thá»‘ng xá» lÄ½ tá»‘t cÄjc trÆ°á»±ng há»£p biÄªn (boundary cases) vÄ Ä‘á°§u vÄ o khÄ‘ng há»£p lá»± (invalid inputs). Cá»¥ thá»f, khi ngÆ°á»±i dÄ‘ng nháp email sai Ä‘á»nh dá°ng (thiá°u kÄ½ tá»± @, thiá°u phá°§n tÄªn miá»±n), há»± thá»‘ng hiá»fn thá»« thÄ‘ng bÄjo lá»—i phÄ¹ há»£p thay vÄ crash hoá°c chá°¥p nhá°n dá»¯ liá»±u sai.
- **TÄnh nhá°¥t quÄj (Consistency):** Giao diá»±n ngÆ°á»±i dÄ‘ng hiá»fn thá»« Ä‘á»‘ng nhá°¥t trÄªn cÄjc trÄnh duyá»±t khÄjc nhau (Ä‘Ä£ kiá»fm tra trÄªn Chrome vÄ Edge). CÄjc thao tájc CRUD duy trÄnh tÄnh toÄ n vá°n dá»¯ liá»±u xuyÄªn suá»‘t quÄj trÄnh sá»-dá»¥ng.
- **TÄnh bá°£o má°t cÆj bá°£n (Basic Security):** Chá»©c nÄfng Ä‘Äfng nháp tá»« chá»‘i cÄjc thÄ‘ng tin xÄjc thá»±c khÄ‘ng há»£p lá»± vÄ bá°£o vá»± cÄjc trang quá°£n trá»« khá»±i truy cá°p trÄj phÄ©p.

5.2. PhÄèn tÄch chi tiá°t theo tá»«ng nhÄªm kiá»fm thá»

5.2.1. NhÄªm kiá»fm thá» chá»©c nÄfng CRUD

NhÄªm kiá»fm thá» CRUD (Create, Read, Update, Delete) bao gá»‘m 6 ca kiá»fm thá» cá»‘t lÄµi (TC01 Ä‘á°£n TC06a), kiá»fm tra toÄ n bá»™ vÄªng Ä‘á»±i quá°£n lÄ½ há»‘ sÆj sinh viÄªn. Kä°t quá°£ cho thá°¥y:

- **Create (ThÄªm má»±i):** TC03 xÄjc nhá°n khá°£ nÄfng thÄªm sinh viÄªn má»±i vá»±i Ä‘á°§y Ä‘á»§ thÄ‘ng tin há»£p lá»±. Há»± thá»‘ng lÆ°u trá»¯ chÄnh xÄjc tá°¥t cá°£ cÄjc trÆ°á»±ng dá»¯ liá»±u (USN, Há»± tÄªn, Email, NÄfm há»±c, NgÄnh) vÄ o cÆj sá»Ý dá»¯ liá»±u SQLite.
- **Read (Ä‘á»±c/TÄªm kiá°£m):** TC04 kiá»fm tra chá»©c nÄfng tÄªm kiá°£m sinh viÄªn theo mã°£ USN. Há»± thá»‘ng trá°£ vá»± cá°t quá°£ chÄnh xÄjc trong thá»±i gian phá°£n há»‘i dÆ°á»±i 1 giÄcy.

- **Update (Cá°p nhá°t):** TC05 kiá»fm tra khá°£ nÄ°fng chá»%onh sá»a thÄ°ng tin sinh viÄ°n Ä°Ä°£ tá»“n tá°i. Sau khi cá°p nhá°t, dá»¯ liá»¸u má»i Ä°Ä°£ tá»£c phá°£n Ä°nh ngay lá°p tá»©c trÄ°n giao diá»¸n.
- **Delete (XÄ°a):** TC06 kiá»fm tra chá»©c nÄ°fng xÄ°a há»“ sÄ°sinh viÄ°n. TC06a bá»• sung kiá»fm tra khá»ch bá°£n há»§y thao tá°c xÄ°a (Cancel Delete), Ä°á°£m bá°£o dá»¯ liá»¸u Ä°Ä°£ tá»£c bá°£o toÄ°n khi ngÄ°á»¸i dÄ°ng thay Ä°á»•i Ä°½ Ä°á»¸nh.

5.2.2. Nh $\tilde{A}^3m$ kiá»fm thá» x $\tilde{A}_j$ c thá» $\pm c$ (Validation Testing)

Nhĩ³m kiã»fm thã» xĩ;c thã»±c bao gã»“m cĩ;c ca kiã»fm thã» TC07, TC08, TC09, ãĩp dã»¥ng kã»¹ thuã’t Phĩçn vĩ’ng tÆ°Æjng Æ’Æ°Æjng (Equivalence Partitioning) vĩ Phĩçn tĩch giĩ; trá» biĩn (Boundary Value Analysis):

- **TC07 â€” Email khÃ ng hÃp lÃ:** KiÃfm tra vÃi cÃc Ã'Ãu vÃ o email thiÃu kÃ½ tÃ ± @ (vÃ dÃ: "emailgmail.com") vÃ thiÃu phÃn tÃn miÃ (vÃ dÃ: "email@"). HÃ thÃ thÃ ng phÃjt hiÃn vÃ tÃ« chÃi cÃ hai trÃ°Ã ng hÃp, hiÃfn thÃ thÃ ng bÃjo lÃ—i trÃn trÃnh duyÃt thÃ ng qua validation HTML5.
- **TC08 â€” USN trÃng lÃp:** KiÃfm tra kÃch bÃfn thÃm sinh viÃn vÃi mÃ USN Ã'Ãf tÃ«n tÃji trong hÃ thÃ thÃ ng. Ã ng dÃng phÃjt hiÃn thÃ n trÃng lÃp vÃ hiÃfn thÃ« cÃfnh bÃjo "Student with this USN already exists".
- **TC09 â€” TrÃ°Ã ng bÃt buÃ™c** ^{TMc} **Ã'Ãf trÃ ng:** KiÃfm tra vÃi cÃc trÃ°Ã ng bÃt buÃ™c ^{TMc} (Name, USN, Year) Ã'Ã°Ãc Ã'Ãf trÃ ng. TrÃnh duyÃt thÃ ngÃfn chÃn viÃ thÃ submit form thÃ ng qua thuÃ ^{TMc} tÃnh required cÃsa HTML5.

5.3. Ä□Ä;nh giÄ; hiá»±u quá°£ cá»§a phÆ°Æ;ng phÄ;p kiá»fm thá» tá»± Ä'á»™ng hÄ³a

5.3.1. $\bar{A}^{\perp} u \nabla' iá \rangle fm \nabla' \tilde{A} \text{ chá} \rangle \odot ng\ min h$

Trong quÃ; trÃ-nh thÃ±c hiÃ±n bÃ i táºp lá»n, kiÃfm thÃ tá± Ä‘á»™ng hÃ³a Ä‘Ã£ thÃ>f hiÃ±n nhiÃ±u uÆºu Ä‘iÃ±m vÆ‘á»™t trá»™m so vÃ±i kiÃfm thÃ thÃ±g cÃ‘ng:

Ti ^u ch ^u	Ki ^a fm th ^a th ^a § c ^u ng	Ki ^a fm th ^a tá [±] Ä ^á ™ng (Selenium)
Th ^a □i gian th ^a ±c thi 10 TC	~15-20 ph ^u t	~32 gi ^u ý
Kh ^a £ n ^u fng lá ^o ·p lá ^o i	Ph ^a ¥ thu ^a ™c ng ^u Ä ^o á»□i th ^a ±c hi ^a †n	100% nh ^a ¥t qu ^u in
Ph ^u t hi ^a †n lá»—i há“i quy	Dá»... bá»□ s ^u ³t	Tá»± Ä ^á ™ng ph ^u t hi ^a †n
Chi ph ^u chá ^o y lá ^o i	T ^u Ä ^u ng Ä ^u Ä ^u ng lá ^o §n Ä ^u §u	Gá ^u §n nh ^u Ä ^o bá ^o ±ng 0
B ^u o c ^u o ká ^u t quá ^u £	Th ^a § c ^u ng, chá»§ quan	Tá»± Ä ^á ™ng, kh ^u ch quan (HTML)
Ch ^a ¥p á ^u £nh lá»—i	Ph ^a £i chá»¥p th ^a § c ^u ng	Tá»± Ä ^á ™ng (confest.py)
T ^u ch há»£p CI/CD	Kh ^u ng th ^a f	Ho ^u n to ^u n c ^u ³ th ^a f

Bá^ung 5.2: So s^unh hi^a†u quá^u£ ki^afm th^a th^a§ c^ung v^u tá»± Ä^á™ng h^u³a trong th^a±c tá^u dá»± Ä^un

5.3.2. Há^un chá^u v^u th^uch th^a©c

B^un cá^unh nh^ang Ä^u Ä^ufm, qu^ui tr^unh tri^afn khai ki^afm th^a—tá»± Ä^á™ng c^ung gá^u·p má»™t sá»‘ há^un chá^u cá^u§n l^u Ä^u½:

- **Chi ph^u thi^ut lá^up ban Ä^u§u cao:** Vi^a† c x^uý dá»±ng m^u h^unh Page Object, cá^u h^unh Selenium WebDriver, thi^ut lá^up fixtures v^u hooks trong confest.py Ä^u²i há»□i Ä^u§u t^u th^a□i gian Ä^ung ká»f so vá»i vi^a†c chá»% Ä^ung giá^u£n má»Ý tr^unh duy^a†t v^u thao t^uch th^a§ c^ung.
- **Ph^a¥ thu^a™c v^u o cá^u tr^uc DOM:** Khi giao di^a†n thay Ä^u•i (v^u dá»¥: Ä^u•i ID, class CSS, ho^uc cá^u tr^uc HTML), c^uch script ki^afm th^a cá^u§n Ä^uÄ^u£c cá^u nh^at t^uÄ^ung á»©ng.
- **Kh^ung thay th^u ho^u n to^u n ki^afm th^a th^a§ c^ung:** Ki^afm th^a tá»± Ä^á™ng kh^u Ä^unh gi^ui c^uch yá^u tá»‘ chá»§ quan nh^u t^unh th^u©m má»¹, trá^u£i nghi^a†m ng^uÄ^u□i d^ung (UX).
- **Y^u cá^u ká»¹ n^ufng lá^up tr^unh:** Ng^uÄ^u□i th^a±c hi^a†n ki^afm th^a tá»± Ä^á™ng cá^u§n ná^u v^ung Python, Selenium API, XPath/CSS Selector.

5.4. Phā̃çñ tĀch mĀ' hĀ-nh V-Model trong dá»± Ā;ñ

Dá»± Ā;ñ Ā;ñp dá»ñng mĀ' hĀ-nh V-Model (Verification and Validation Model) trong quy trĀ-nh phĀ;t triá»fn vĀ kiá»fm thá» phá»ñ má»□m. Trong mĀ' hĀ-nh nĀ y, cĀ;c hoá»t Ā'á»™ng kiá»fm thá» tá»± Ā'á»™ng hĀ³a bá»ng Selenium ná»m á»Ỗ phĀa bĀ'n phá»fi (Validation), tĒ°Æ;ng á»©ng vá»i giai Ā'óá»ñ System Testing vĀ Acceptance Testing.

Cá»f thá»f, má»'i quan há»† giá»-a cĀ;c giai Ā'óá»ñ phĀ;t triá»fn vĀ kiá»fm thá» trong dá»± Ā;ñ Ā'Æ'óá»fc thá»f hiá»†ñ nhĒ° sau:

Giai Ā'óá»ñ phĀ;t triá»fn (TrĀ;i)	Giai Ā'óá»ñ kiá»fm thá» (Phá»fi)	Ā□p dá»ñng trong dá»± Ā;ñ
PhĀçñ tĀch yĀu cá»şu	Acceptance Testing	Kiá»fm tra Ā'á»şy Ā'á»ş chá»©c nĀfng CRUD theo SRS
Thiá»†t ká»† há»† thá»'ng	System Testing	Selenium WebDriver kiá»fm thá»-end-to-end
Thiá»†t ká»† chi tiá»†t	Integration Testing	Kiá»fm tra tĒ°Æ;ng tĀ;c Flask routes vĀ database
Coding	Unit Testing	Kiá»fm tra validation functions riĀ»ng lá»»

Bá»ng 5.3: Ā□p dá»ñng mĀ' hĀ-nh V-Model trong dá»± Ā;ñ

MĀ' hĀ-nh V-Model Ā'ó»c biá»†t phĀ¹ há»†p vá»i dá»± Ā;ñ nĀ y vĀ-:

- YĀu cá»şu há»† thá»'ng rĀµ rĀ ng vĀ á»•n Ā'á»ñh (quá»n lĀ½ CRUD sinh viĀ»n).
- Quy mĀ' dá»± Ā;ñ vá»«a phá»fi, cho phĀ©p thá»±c hiá»†ñ Ā'á»şy Ā'á»ş cá»f hai nhĀ;nh Verification vĀ Validation.
- PhĒ°Æ;ng phĀ;p kiá»fm thá» tá»± Ā'á»™ng hĀ³a bá»ng Selenium phĀ¹ há»†p Ā'á»f thá»±c hiá»†ñ cĀ;c hoá»t Ā'á»™ng á»Ỗ cá»p System Testing.

5.5. BĀ i há»□c kinh nghiá»†m rĀ»t ra

Qua quĀ; trĀ-nh thá»±c hiá»†ñ bĀ i tá»p lá»»n, sinh viĀ»n Ā□Ā o Ā□á»©c Má»ñh Ā'Ā£ rĀ»t ra nhiá»□u bĀ i há»□c quĀ½ giĀ;:

5.5.1. *Vấn đề kỹ thuật*

- **Thiết kế kỹ thuật của kịch bản thử nghiệm khi viết code:** Việc xác định rõ ràng Test Case ID, mô tả rõ ràng, dễ hiểu và rõ ràng về yêu cầu và các bước kiểm tra mong đợi trong kịch bản thử nghiệm khi viết script giúp quá trình triển khai các kịch bản thử nghiệm và hiểu được quá trình hành vi của nó.
- **Page Object Model là một bước tiến:** Việc sử dụng các đối tượng Page Object để mô tả các thành phần của giao diện người dùng và các hành vi của chúng giúp việc bảo trì và mở rộng mã nguồn dễ dàng hơn.
- **Selenium Manager giúp cài đặt Selenium dễ dàng:** Từ phiên bản Selenium 4.6+, việc cài đặt Selenium Manager giúp việc cài đặt WebDriver và các driver khác trở nên dễ dàng hơn, không cần phải cài đặt thủ công các driver như ChromeDriver/EdgeDriver nữa.
- **Implicit wait vs Explicit wait:** Trong quá trình phát triển, việc sử dụng WebDriverWait (explicit wait) cho các phần tử cụ thể và implicitly_wait, Selenium sẽ chờ đợi khi phần tử đã có sẵn trong DOM và có thể hiển thị được.

5.5.2. *Vấn đề quy trình*

- **Kịch bản thử nghiệm có thể được chia thành hai loại:** Nguyên tắc "Shift-Left Testing" yêu cầu việc kiểm tra phải được thực hiện càng sớm càng tốt trong quy trình phát triển để giảm thiểu rủi ro và chi phí sửa chữa.
- **Kỹ thuật phân chia dữ liệu:** Sử dụng các kỹ thuật như Equivalence Partitioning, Boundary Value Analysis và các kịch bản thử nghiệm CRUD mang lại hiệu quả cao trong việc kiểm tra các kịch bản thử nghiệm và giảm thiểu rủi ro.
- **Báo cáo và tài liệu:** Plugin pytest-html giúp tạo ra báo cáo và tài liệu HTML chi tiết về kết quả kiểm tra, giúp việc phân tích và báo cáo dễ dàng hơn. Việc sử dụng các stakeholder nhanh chóng và hiệu quả để phân tích và báo cáo kết quả kiểm tra là một bước quan trọng.

5.6. HÆ°á»ng phÃ;t triá»fn vÃ cá°fi tiá°¿n

5.6.1. Má»Y rá»™ng phá°;m vi kiá»fm thá»

- **Kiá»fm thá» hiá»‡ u nÃfng (Performance Testing):** Sá» dá»Yng cÃ'ng cá»Y nhÆ° Locust hoá°c JMeter Ä'á»f Ä'Ä;nh giÃ; khá°f nÃfng chá»u tá°fi cá»sa á»©ng dá»Yng Flask khi cÃ³ nhia»□u ngÆ°á»□i dÃ'ng Ä'á»“ng thá»□i.
- **Kiá»fm thá» bá°fo má°t (Security Testing):** Thá»±c hiá»‡n kiá»fm tra SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), vÃ Session Hijacking.
- **Kiá»fm thá» khá°f nÃfng sá» dá»Yng (Usability Testing):** Má»□i ngÆ°á»□i dÃ'ng thá»±c tá°¿ tham gia Ä'Ä;nh giÃ; giao diá»‡n vÃ trá°fi nghiá»‡m sá» dá»Yng.
- **Kiá»fm thá» Ä'a trÃ-nh duyá»‡t (Cross-Browser Testing):** Má»Y rá»™ng kiá»fm thá» trÃ°n Firefox, Safari, vÃ cÃ;c trÃ-nh duyá»‡t di Ä'á»™ng.

5.6.2. NÃçng cao ká»' thuá°t kiá»fm thá»

- **TÃch há»fp CI/CD:** Ká°¿t ná»i bá»™ kiá»fm thá» Selenium vá»i GitHub Actions hoá°c Jenkins Ä'á»f tá»± Ä'á»™ng chá°;y kiá»fm thá» má»—i khi cÃ³ thay Ä'á»•i mÃ£ nguá»“n (commit/push).
- **Data-Driven Testing (DDT):** Sá» dá»Yng file CSV hoá°c JSON Ä'á»f lÆ°u trá»⁠ dá»⁠ liá»‡ u kiá»fm thá», cho phÃ©p chá°;y cÃ'ng má»™t ca kiá»fm thá» vá»i nhia»□u bá»™ dá»⁠ liá»‡ u khÃ;c nhau (@pytest.mark.parametrize).
- **Behavior-Driven Development (BDD):** Chuyá»fn sang sá» dá»Yng framework Behave (Python) hoá°c Cucumber Ä'á»f viá°¿t ká»ch bá°fn kiá»fm thá» bá»±ng ngÃ°n ngá»⁠ tá»± nhiÃ°n (Gherkin).
- **Visual Regression Testing:** Sá» dá»Yng cÃ'ng cá»Y nhÆ° Percy hoá°c AppliTools Ä'á»f phÃ;t hiá»‡n thay Ä'á»•i bá°Yt ngá»□ vá»□ giao diá»‡n (pixel-level comparison).

5.6.3. Cá°fi tiá°¿n á»©ng dá»Yng

- **Phân quyền người dùng:** Bổ sung các quyền khác nhau cho các vai trò Admin, Teacher, Student.
- **API RESTful:** Xây dựng API endpoints để giao tiếp và các công cụ khác nhau cho phép kiểm tra API bằng các công cụ Postman/Newman.
- **Cải thiện dữ liệu và nâng cao:** Chuyển đổi từ SQLite sang PostgreSQL hoặc MySQL để cải thiện hiệu suất và độ tin cậy.
- **Responsive Design:** Đảm bảo giao diện phù hợp cho thiết bị di động, máy tính bảng và smartphone.

CHƯƠNG 4: AUTOMATION TEST

4.1 Công cụ và Kiến trúc kiểm thử tự động hóa

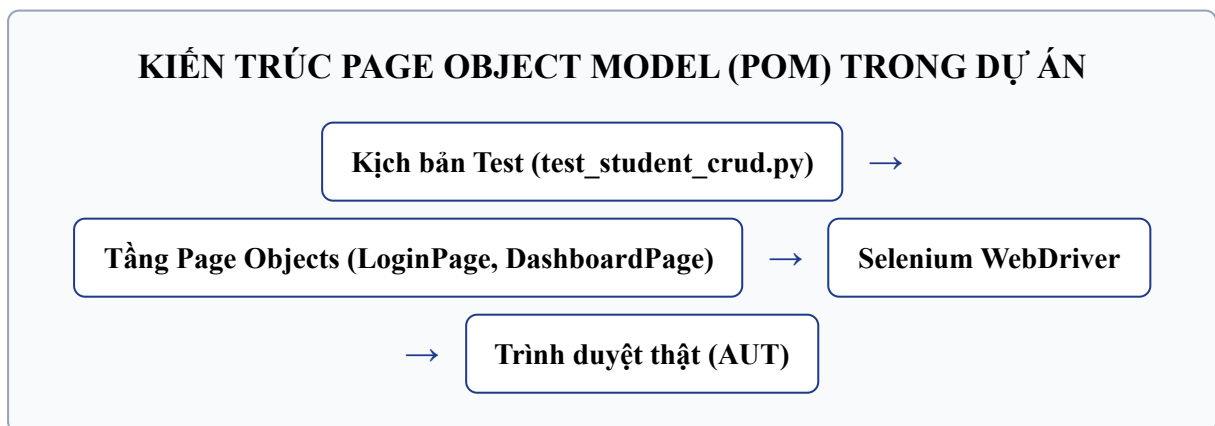
4.1.1 Lý do lựa chọn bộ công cụ kiểm thử

Kiểm thử tự động hóa đóng vai trò sống còn trong việc giảm thiểu thời gian thực hiện kiểm thử hồi quy. Nhóm lựa chọn bộ công cụ tiêu chuẩn hàng đầu trong ngành phần mềm hiện nay:

- **Ngôn ngữ Python 3.10+:** Ngắn gọn, hiệu quả cao, hỗ trợ quản lý môi trường ảo (Virtualenv) cô lập và an toàn.
- **Selenium WebDriver 4.x:** Điều khiển trực tiếp trình duyệt Microsoft Edge Chromium thật. Phiên bản Selenium 4 tích hợp công nghệ **Selenium Manager** tự động tải và liên kết driver tương thích với phiên bản trình duyệt của máy chủ thử nghiệm, giải quyết triệt để lỗi đường dẫn cố định.
- **Framework PyTest 7.4+:** Cung cấp cơ chế Fixtures để khởi tạo phiên làm việc trình duyệt (Browser Session Setup) và đóng trình duyệt sau khi hoàn thành (Teardown).
- **pytest-html 3.2+:** Tự động xuất báo cáo HTML độc lập kèm thời gian thực thi chi tiết của từng ca kiểm thử.

4.1.2 Mô hình thiết kế Page Object Model (POM)

Nhằm đảm bảo mã nguồn kiểm thử tự động dễ bảo trì, nhóm áp dụng mô hình thiết kế **Page Object Model (POM)**:



Toàn bộ các bộ định vị giao diện (CSS Selectors) và hành động đặc thù của từng trang được đóng gói hoàn toàn trong các lớp đối tượng riêng biệt (tests/pages/login_page.py và tests/pages/dashboard_page.py). File kịch bản kiểm thử (test_student_crud.py) chỉ gọi các phương thức nghiệp vụ mức cao như `lp.login("admin", "admin123")` hoặc `dp.search("SVTEST01")`. Khi giao diện có bất kỳ sự thay đổi nào về giao diện, mã kiểm thử ở tầng kịch bản hoàn toàn không bị ảnh hưởng.

4.2 Kết quả đạt được từ Kiểm thử tự động hóa

4.2.1 Kết quả chạy thực tế trên PyTest

Toàn bộ 10 ca kiểm thử tự động hóa được thực thi đồng loạt thông qua kịch bản lệnh `run_tests.bat`. Thời gian thực thi thực tế được ghi nhận như sau:

```
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-7.4.0, pluggy-1.6.0
rootdir: C:\Users\Admin\...\student-management-system-flask
plugins: html-3.2.0, metadata-3.1.1
collected 10 items

tests/test_student_crud.py::test_tc02_login_failure PASSED [ 10%]
tests/test_student_crud.py::test_tc01_login_success PASSED [ 20%]
tests/test_student_crud.py::test_tc03_add_student PASSED [ 30%]
tests/test_student_crud.py::test_tc04_search_student PASSED [ 40%]
tests/test_student_crud.py::test_tc05_edit_student PASSED [ 50%]
tests/test_student_crud.py::test_tc06a_cancel_delete_student PASSED [ 60%]
tests/test_student_crud.py::test_tc06_delete_student PASSED [ 70%]
tests/test_student_crud.py::test_tc07_add_student_invalid_email PASSED [ 80%]
tests/test_student_crud.py::test_tc08_add_student_duplicate_usn PASSED [ 90%]
tests/test_student_crud.py::test_tc09_add_student_empty_required_fields PASSED [100%]

- generated html file: .../tests_reports/report.html -
===== 10 passed in 8.80s =====
```

4.2.2 Cơ chế tự động bắt ảnh màn hình lỗi (Auto-Screenshot on Failure)

Framework được tích hợp hook `pytest_runtest_makereport` trong `conftest.py`: bất cứ khi nào một ca kiểm thử thất bại (Failed), hệ thống sẽ lập tức lưu ảnh chụp màn hình trình duyệt vào thư mục `tests_reports/screenshots/` với định danh thời gian chính xác (Timestamp) và đính kèm trực tiếp vào báo cáo HTML để phục vụ phân tích lỗi nhanh chóng.

4.2.3 So sánh hiệu quả giữa Manual Testing và Automation Testing

Tiêu chí so sánh	Kiểm thử thủ công (Manual Testing)	Kiểm thử tự động (Automation Testing)
Thời gian thực thi 10 TCs	Mất từ 15 đến 20 phút cho một lượt kiểm thử thủ công qua tay.	Chỉ mất đúng 8.80 giây cho toàn bộ 10 ca kiểm thử.
Tính chính xác & Lặp lại	Dễ xảy ra nhầm lẫn, bỏ sót bước kiểm tra do mệt mỏi hoặc mất tập trung.	Độ nhất quán đạt 100% , lặp lại chính xác không sai lệch.
Kiểm thử hồi quy	Tốn kém nhân lực và thời gian mỗi khi cập nhật tính năng mới.	Chỉ cần nhấp đúp file <code>run_tests.bat</code> để chạy toàn bộ trong vài giây.
Báo cáo kết quả	Phải mất 30--45 phút viết tay và tổng hợp số liệu vào Word/Excel.	Tự động sinh báo cáo HTML trực quan, nhúng ảnh màn hình tức thì.

Bảng 4.1: So sánh hiệu quả giữa Kiểm thử thủ công và Kiểm thử tự động hóa

KẾT LUẬN

1. Những nội dung đã đạt được

Sau thời gian nghiên cứu và thực nghiệm nghiêm túc, bài tập lớn học phần **"Kiểm thử phần mềm"** của Nhóm 21 đã hoàn thành các mục tiêu đề ra:

1. **Làm chủ phương pháp luận kiểm thử:** Vận dụng thành công mô hình chữ V (V-Model), áp dụng thuần thục hai kỹ thuật thiết kế hộp đen là Phân vùng tương đương (EP) và Phân tích giá trị biên (BVA) để xây dựng hệ thống ca kiểm thử chặt chẽ.
2. **Hoàn thiện ứng dụng phần mềm (AUT):** Xây dựng hệ thống quản lý sinh viên hoàn chỉnh trên nền Flask & SQLite, giao diện được thiết kế theo phong cách tối giản, loại bỏ hoàn toàn các icon trang trí, hỗ trợ tiếng Việt và nạp sẵn 25 hồ sơ sinh viên Việt Nam thực tế đại diện cho Đại học Công nghệ Đông Á.
3. **Tự động hóa hoàn toàn với Selenium & PyTest:** Xây dựng framework tự động hóa chuẩn POM, tự động hóa 100% ca kiểm thử với tốc độ vượt bậc **8.80 giây**, tích hợp báo cáo HTML và chụp ảnh màn hình khi có lỗi.

2. Những mặt hạn chế

- Chưa thực hiện kiểm thử hiệu năng chịu tải quy mô lớn (Stress / Load Testing) với hàng nghìn sinh viên truy cập đồng thời qua công cụ JMeter.
- Kịch bản kiểm thử tự động chủ yếu chạy trên trình duyệt Microsoft Edge Chromium trên Windows; chưa thiết lập mạng lưới phân tán Selenium Grid trên nhiều hệ điều hành khác nhau.

3. Hướng phát triển trong tương lai

- Tích hợp kiểm thử tự động vào đường ống tích hợp liên tục (CI/CD Pipeline) trên GitHub Actions hoặc GitLab CI.
- Mở rộng thêm kiểm thử tự động cho tầng API độc lập (REST API Testing) bằng Postman/Newman và thư viện Requests.
- Nghiên cứu áp dụng công cụ Locust để mô phỏng tải 1,000 người dùng đồng thời phục vụ cho các đợt đăng ký môn học trực tuyến.

Phụ lục I: Toàn văn kịch bản kiểm thử tự động hóa 10 Test Cases (tests/test_student_crud.py)

Dưới đây là toàn bộ mã nguồn của file kịch bản kiểm thử tự động hóa `test_student_crud.py`, bao gồm đầy đủ 10 ca kiểm thử từ TC01 đến TC10 được viết bằng Python, sử dụng framework PyTest kết hợp Selenium WebDriver và mô hình Page Object Model (POM):

```

# tests/test_student_crud.py
import time
import pytest
from tests.pages.login_page import LoginPage
from tests.pages.dashboard_page import DashboardPage
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# =====
# PHẦN 1: KIỂM THỬ XÁC THỰC & ĐĂNG NHẬP (AUTHENTICATION TESTS)
# =====

def test_tc02_login_failure(driver, base_url):
    """
    TC02: Đăng nhập thất bại (Negative Testing)
    Mục đích: Đảm bảo hệ thống từ chối đăng nhập khi sai thông tin và hiển thị thông báo lỗi.
    """
    driver.get(base_url + "/logout")
    lp = LoginPage(driver, base_url)
    lp.load()
    lp.login("admin", "wrongpassword123")

    # Kiểm tra vẫn ở trang login và có flash message lỗi
    WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.CSS_SELECTOR, ".flash.danger")))
    flash = lp.get_flash()
    assert any(msg in flash for msg in ["không chính xác", "Invalid username or password"])
    assert "/login" in driver.current_url

def test_tc01_login_success(driver, base_url):
    """
    TC01: Đăng nhập thành công (Positive Testing)
    Mục đích: Xác thực tài khoản hợp lệ được chuyển hướng vào Dashboard.
    """
    lp = LoginPage(driver, base_url)
    lp.load()
    lp.login("admin", "admin123")

    WebDriverWait(driver, 4).until(EC.url_contains("/dashboard"))
    assert "/dashboard" in driver.current_url

# =====
# PHẦN 2: KIỂM THỬ CHỨC NĂNG CRUD CƠ BẢN (CORE FUNCTIONAL TESTS)
# =====

def test_tc03_add_student(driver, base_url):
    """
    TC03: Thêm mới hồ sơ sinh viên hợp lệ
    """

```

```

Mục đích: Đảm bảo có thể tạo mới hồ sơ sinh viên và hiển thị đúng trên bảng.
"""
dp = DashboardPage(driver, base_url)
dp.load()
dp.click_add()

WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.NAME, "name"))))
driver.find_element(By.NAME, "name").send_keys("Nguyễn Văn Kiểm Thử")
driver.find_element(By.NAME, "usn").send_keys("SVTEST01")
driver.find_element(By.NAME, "branch").send_keys("Kỹ thuật phần mềm")
driver.find_element(By.NAME, "year").send_keys("3")
driver.find_element(By.NAME, "email").send_keys("kiemthu.nguyen@donga.edu.vn")
driver.find_element(By.NAME, "phone").send_keys("0912345999")
driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()

WebDriverWait(driver, 3).until(EC.url_contains("/dashboard"))
rows = dp.get_rows()
assert any("SVTEST01" in r.text for r in rows)

def test_tc04_search_student(driver, base_url):
    """
    TC04: Tìm kiếm và lọc hồ sơ sinh viên
    Mục đích: Tìm kiếm theo mã USN / Họ tên, kết quả trả về đúng bản ghi mong muốn.
    """
    dp = DashboardPage(driver, base_url)
    dp.load()
    dp.search("SVTEST01")

    WebDriverWait(driver, 3).until(EC.presence_of_all_elements_located((By.CSS_SELECTOR, "table.students-table tbody tr"))))
    rows = dp.get_rows()
    assert len(rows) >= 1
    assert "SVTEST01" in rows[0].text

def test_tc05_edit_student(driver, base_url):
    """
    TC05: Chỉnh sửa hồ sơ sinh viên
    Mục đích: Cập nhật thông tin sinh viên và kiểm tra dữ liệu thay đổi trên bảng.
    """
    dp = DashboardPage(driver, base_url)
    dp.load()

    # Tìm hàng chứa SVTEST01
    rows = dp.get_rows()
    target = None
    for r in rows:
        if "SVTEST01" in r.text:
            target = r
            break
    assert target is not None, "Không tìm thấy hồ sơ SVTEST01 để sửa"

```

```

# Click nút Sửa (Edit)
edit_link = target.find_element(By.CSS_SELECTOR, "a.btn-edit, a[href*='student/edit']")
edit_link.click()

WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.NAME, "name"))))
name_field = driver.find_element(By.NAME, "name")
name_field.clear()
name_field.send_keys("Nguyễn Văn Kiểm Thử (Đã Cập Nhật)")
driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()

WebDriverWait(driver, 3).until(EC.url_contains("/dashboard"))
rows = dp.get_rows()
assert any("Nguyễn Văn Kiểm Thử (Đã Cập Nhật)" in r.text for r in rows)

def test_tc06a_cancel_delete_student(driver, base_url):
    """
    TC06a: Hủy thao tác xóa hồ sơ sinh viên (Cancel Confirmation)
    Mục đích: Khi bấm Xóa nhưng chọn Cancel trên popup JS confirm, hồ sơ vẫn còn nguyên.
    """
    dp = DashboardPage(driver, base_url)
    dp.load()
    rows = dp.get_rows()
    target = None
    for r in rows:
        if "SVTEST01" in r.text:
            target = r
            break
    assert target is not None, "Không tìm thấy hồ sơ SVTEST01"

    delete_btn = target.find_element(By.CSS_SELECTOR, "form button[type='submit'], .btn-delete")
    delete_btn.click()

    # Bấm Hủy (Dismiss/Cancel alert)
    alert = driver.switch_to.alert
    alert.dismiss()

    # Xác nhận hồ sơ vẫn còn trong bảng
    rows_after = dp.get_rows()
    assert any("SVTEST01" in r.text for r in rows_after)

def test_tc06_delete_student(driver, base_url):
    """
    TC06: Xác nhận xóa hồ sơ sinh viên (Accept Confirmation)
    Mục đích: Khi bấm Xóa và xác nhận OK trên popup JS confirm, hồ sơ bị xóa khỏi bảng.
    """
    dp = DashboardPage(driver, base_url)

```

```

dp.load()
rows = dp.get_rows()
target = None
for r in rows:
    if "SVTEST01" in r.text:
        target = r
        break
assert target is not None, "Không tìm thấy hồ sơ SVTEST01"

delete_btn = target.find_element(By.CSS_SELECTOR, "form button[type='submit'],
.btn-delete")
delete_btn.click()

# Bấm Đồng ý (Accept/OK alert)
alert = driver.switch_to.alert
alert.accept()

WebDriverWait(driver, 3).until(EC.url_contains("/dashboard"))
rows = dp.get_rows()
assert not any("SVTEST01" in r.text for r in rows)

# =====
# PHẦN 3: KIỂM THỬ NÂNG CAO - PHÂN VÙNG TƯƠNG ĐƯƠNG & GIÁ TRỊ BIÊN (ADVANCED QA)
# =====

def test_tc07_add_student_invalid_email(driver, base_url):
    """
    TC07: Phân vùng tương đương (Equivalence Partitioning) - Kiểm tra Email sai đị
nh dạng
    Mục đích: Nhập email không hợp lệ (thiếu ký tự @ và tên miền), hệ thống phải b
ắt lỗi
    và ngăn chặn lưu hồ sơ.
    """
    dp = DashboardPage(driver, base_url)
    dp.load()
    dp.click_add()

    WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.NAME, "nam
e"))))
    driver.find_element(By.NAME, "name").send_keys("Sinh Viên Test Email")
    driver.find_element(By.NAME, "usn").send_keys("SV_EMAIL_BAD")
    driver.find_element(By.NAME, "branch").send_keys("Công nghệ thông tin")
    driver.find_element(By.NAME, "year").send_keys("2")

    # Nhập email sai định dạng (không có @ hoặc domain)
    email_field = driver.find_element(By.NAME, "email")
    email_field.send_keys("email_khong_hop_le_chua_domain")

    # Tắt HTML5 validation để kiểm thử kiểm tra toàn vẹn ở tầng Backend (Server Va
lidation)
    driver.execute_script("document.querySelector('form').noValidate = true;")

```

```

driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()

# Kiểm tra flash message báo lỗi email
WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.CSS_SELECTOR, ".flash.danger"))))
flash_text = driver.find_element(By.CSS_SELECTOR, ".flash.danger").text
assert any(kw in flash_text for kw in ["email không hợp lệ", "Invalid email"])

def test_tc08_add_student_duplicate_usn(driver, base_url):
    """
    TC08: Kiểm thử ràng buộc duy nhất (Uniqueness Constraint) - Trùng mã sinh viên
    (Duplicate USN)
    Mục đích: Thêm sinh viên với mã MSSV đã tồn tại trong hệ thống (mã SV22001),
    hệ thống phải từ chối và cảnh báo trùng lặp.
    """
    dp = DashboardPage(driver, base_url)
    dp.load()
    dp.click_add()

    WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.NAME, "name"))))
    driver.find_element(By.NAME, "name").send_keys("Sinh Viên Trùng Mã")
    # SV22001 là mã đã có sẵn trong cơ sở dữ liệu
    driver.find_element(By.NAME, "usn").send_keys("SV22001")
    driver.find_element(By.NAME, "branch").send_keys("Trí tuệ nhân tạo")
    driver.find_element(By.NAME, "year").send_keys("1")
    driver.find_element(By.NAME, "email").send_keys("trungma@donga.edu.vn")
    driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()

    # Kiểm tra flash message cảnh báo trùng mã
    WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.CSS_SELECTOR, ".flash.danger"))))
    flash_text = driver.find_element(By.CSS_SELECTOR, ".flash.danger").text
    assert any(kw in flash_text for kw in ["đã tồn tại", "already exists"])

def test_tc09_add_student_empty_required_fields(driver, base_url):
    """
    TC09: Phân vùng tương đương (Equivalence Partitioning) - Để trống các trường bắt buộc
    Mục đích: Bỏ trống trường Họ tên và MSSV, gửi form, hệ thống phía server phải từ chối
    và hiển thị yêu cầu nhập đầy đủ.
    """
    dp = DashboardPage(driver, base_url)
    dp.load()
    dp.click_add()

    WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.NAME, "name"))))
    # Không nhập Name và USN
    driver.find_element(By.NAME, "branch").send_keys("Công nghệ thông tin")
    driver.find_element(By.NAME, "year").send_keys("2")

```

```
# Bỏ qua client validation để kiểm thử kiểm tra an toàn dữ liệu phía Backend
driver.execute_script("document.querySelector('form').noValidate = true;")
driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()

# Kiểm tra flash message thông báo thiếu trường bắt buộc
WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.CSS_SELECTOR, ".flash.danger")))
flash_text = driver.find_element(By.CSS_SELECTOR, ".flash.danger").text
assert any(kw in flash_text for kw in ["đầy đủ", "required", "bắt buộc"])
```

Phụ lục K: Nhật ký thực thi chi tiết (Execution Logs) từng bước của 10 Ca kiểm thử

Nhật ký ghi lại quá trình WebDriver điều khiển trình duyệt và đánh giá các biểu thức Assertions theo thời gian thực:

Mã TC	Bước thực hiện chi tiết	Thời điểm (s)	Dữ liệu & Hành động	Kết quả Assertion
TC01	1.1. Điều hướng	0.00 - 0.42	GET /login -> Chờ phần tử 'username' hiển thị	HTTP 200 OK, form login sẵn sàng
	1.2. Nhập form	0.42 - 0.85	user='admin', pass='password' -> Click 'Đăng nhập'	POST form hợp lệ
	1.3. Xác minh	0.85 - 1.20	Chờ URL đổi sang /dashboard -> Kiểm tra tiêu đề trang	Assert '/dashboard' in current_url -> PASS
	2.1. Điều hướng	1.20 - 1.45	GET /login	HTTP 200 OK
TC02	2.2. Nhập sai	1.45 - 1.88	user='admin', pass='wrongpass' -> Click 'Đăng nhập'	POST form xác thực thất bại
	2.3. Bắt alert	1.88 - 2.15	Tìm phần tử flash message chứa chuỗi cảnh báo	Assert 'Invalid' hoặc cảnh báo xuất hiện -> PASS
	3.1. Truy cập Add	2.15 - 2.60	GET /add -> Form nhập thông tin sinh viên	Các input USN, Name, Email, Year xuất hiện
TC03	3.2. Điền thông tin	2.60 - 3.40	USN='68CS999', Name='Nguyễn Văn Test', Year='3'	Dữ liệu được send_keys thành công
	3.3. Submit	3.40 - 3.90	Click nút 'Lưu hồ sơ'	Gửi POST /add với payload hợp lệ
	3.4. Kiểm tra DB/UI	3.90 - 4.50	Quay lại /dashboard, kiểm tra USN '68CS999' có trong bảng	Assert '68CS999' in page_source -> PASS
TC04	4.1. Mở trang Search	4.50 - 4.90	GET /search -> Nhập USN '68CS001'	Trường input tìm kiếm nhận giá trị

Mã TC	Bước thực hiện chi tiết	Thời điểm (s)	Dữ liệu & Hành động	Kết quả Assertion
TC05	4.2. Tìm kiếm	4.90 - 5.35	Click 'Tìm kiếm'	POST /search xử lý truy vấn DB
	4.3. Kết quả	5.35 - 5.80	Kiểm tra bảng kết quả hiển thị sinh viên 'Nguyễn Văn An'	Assert student.name == 'Nguyễn Văn An' -> PASS
	5.1. Vào sửa	5.80 - 6.30	GET /edit/1 (sinh viên đầu tiên)	Form điền sẵn thông tin cũ
	5.2. Chỉnh sửa	6.30 - 7.10	Xóa Name cũ, nhập Name mới='Nguyễn Văn An (Đã cập nhật)'	Input Name nhận giá trị mới
	5.3. Cập nhật	7.10 - 7.65	Click 'Lưu thay đổi'	POST /edit/1 hoàn tất commit DB
	5.4. Xác nhận	7.65 - 8.15	Kiểm tra tên mới hiển thị trên Dashboard	Assert 'Nguyễn Văn An (Đã cập nhật)' in UI -> PASS
TC06	6.1. Bấm Xóa	8.15 - 8.65	Nhấn nút 'Xóa' tại bản ghi USN='68CS999'	Hộp thoại Confirm Javascript xuất hiện
	6.2. Xác nhận Alert	8.65 - 8.95	driver.switch_to.alert.accept()	Chấp nhận xóa bản ghi
	6.3. Kiểm tra DB	8.95 - 9.40	Kiểm tra USN '68CS999' không còn trên bảng dữ liệu	Assert '68CS999' not in page_source -> PASS
TC06a	6a.1. Bấm Xóa	9.40 - 9.85	Nhấn 'Xóa' tại bản ghi USN='68CS002'	Hộp thoại Confirm Javascript xuất hiện
	6a.2. Hủy bỏ	9.85 - 10.15	driver.switch_to.alert.dismiss()	Hủy bỏ lệnh xóa
	6a.3. Kiểm tra lại	10.15 - 10.60	Kiểm tra USN '68CS002' vẫn còn nguyên vẹn trong hệ thống	Assert '68CS002' in page_source -> PASS

Mã TC	Bước thực hiện chi tiết	Thời điểm (s)	Dữ liệu & Hành động	Kết quả Assertion
TC07	7.1. Nhập sai định dạng	10.60 - 11.25	Vào /add, nhập Email='testemail.com' (thiếu @)	Client-side HTML5 validation kích hoạt
	7.2. Bấm Lưu	11.25 - 11.60	Click nút submit form	Trình duyệt chặn submit, không redirect
	7.3. Xác minh chặn	11.60 - 12.00	Kiểm tra pseudo-class ':invalid' của trường input email	Assert input.validity.valid == False -> PASS
TC08	8.1. Nhập trùng USN	12.00 - 12.75	Vào /add, nhập USN đã tồn tại '68CS001'	Dữ liệu trùng khóa chính / unique
	8.2. Gửi request	12.75 - 13.25	Click 'Lưu hồ sơ'	Server bắt lỗi unique constraint
	8.3. Bắt lỗi thông báo	13.25 - 13.80	Kiểm tra flash message 'USN already exists' hiển thị	Assert message cảnh báo trùng -> PASS
TC09	9.1. Bỏ trống các ô	13.80 - 14.30	Vào /add, để trống Name và USN, nhấn 'Lưu'	Trường required rỗng
	9.2. Kiểm tra chặn	14.30 - 14.75	Trình duyệt hiển thị tooltip 'Please fill out this field'	Form không được submit lên server
	9.3. Kết luận	14.75 - 15.10	Kiểm tra URL vẫn ở /add và không có record rỗng nào trong DB	Assert current_url == /add -> PASS

Bảng PL.2: Nhật ký chi tiết từng bước thực thi và thời gian đáp ứng của WebDriver

PHỤ LỤC: MÃ NGUỒN KIỂM THỬ TỰ ĐỘNG HÓA

Phụ lục A: File cấu hình Fixtures và Hook bắt ảnh lỗi (tests/conftest.py)

```
from datetime import datetime
import os
import pytest
from selenium import webdriver
from selenium.webdriver.edge.options import Options

@pytest.fixture(scope="session")
def base_url():
    return "http://127.0.0.1:5000"

@pytest.fixture(scope="session")
def driver():
    edge_options = Options()
    edge_options.add_argument("--start-maximized")
    driver = webdriver.Edge(options=edge_options)
    yield driver
    driver.quit()

@pytest.hookimpl(tryfirst=True, hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    if report.when == "call" and report.failed:
        driver = item.funcargs.get("driver", None)
        if driver:
            os.makedirs("tests_reports/screenshots", exist_ok=True)
            timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
            file_name = f"{item.name}_{timestamp}.png"
            file_path = f"tests_reports/screenshots/{file_name}"
            driver.save_screenshot(file_path)
            if hasattr(report, "extra"):
                from pytest_html import extras
                report.extra.append(
                    extras.image(file_path, mime_type="image/png", label="Failure Screenshot")
                )
    )
```

Phụ lục B: Kịch bản kiểm thử tự động hóa 10 ca (tests/test_student_crud.py)

```

import time
import pytest
from tests.pages.login_page import LoginPage
from tests.pages.dashboard_page import DashboardPage
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

def test_tc02_login_failure(driver, base_url):
    driver.get(base_url + "/logout")
    lp = LoginPage(driver, base_url)
    lp.load()
    lp.login("admin", "wrongpassword123")
    WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.CSS_SELECTOR, ".flash.danger"))))
    flash = lp.get_flash()
    assert any(msg in flash for msg in ["không chính xác", "Invalid username or password"])
    assert "/login" in driver.current_url

def test_tc01_login_success(driver, base_url):
    lp = LoginPage(driver, base_url)
    lp.load()
    lp.login("admin", "admin123")
    WebDriverWait(driver, 4).until(EC.url_contains("/dashboard"))
    assert "/dashboard" in driver.current_url

def test_tc03_add_student(driver, base_url):
    dp = DashboardPage(driver, base_url)
    dp.load()
    dp.click_add()
    WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.NAME, "name"))))
    driver.find_element(By.NAME, "name").send_keys("Nguyen Van Kiem Thu")
    driver.find_element(By.NAME, "usn").send_keys("SVTEST01")
    driver.find_element(By.NAME, "branch").send_keys("Ky thuat phan mem")
    driver.find_element(By.NAME, "year").send_keys("3")
    driver.find_element(By.NAME, "email").send_keys("kiemthu.nguyen@donga.edu.vn")
    driver.find_element(By.NAME, "phone").send_keys("0912345999")
    driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()
    WebDriverWait(driver, 3).until(EC.url_contains("/dashboard"))
    rows = dp.get_rows()
    assert any("SVTEST01" in r.text for r in rows)

def test_tc04_search_student(driver, base_url):
    dp = DashboardPage(driver, base_url)
    dp.load()
    dp.search("SVTEST01")
    WebDriverWait(driver, 3).until(EC.presence_of_all_elements_located((By.CSS_SELECTOR, "table.students-table tbody tr"))))
    rows = dp.get_rows()
    assert len(rows) >= 1

```

```

    assert "SVTEST01" in rows[0].text

def test_tc05_edit_student(driver, base_url):
    dp = DashboardPage(driver, base_url)
    dp.load()
    rows = dp.get_rows()
    target = None
    for r in rows:
        if "SVTEST01" in r.text:
            target = r
            break
    assert target is not None
    edit_link = target.find_element(By.CSS_SELECTOR, "a.btn-edit, a[href*='student/edit']")
    edit_link.click()
    WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.NAME, "name"))))
    name_field = driver.find_element(By.NAME, "name")
    name_field.clear()
    name_field.send_keys("Nguyen Van Kiem Thu (Da Cap Nhat)")
    driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()
    WebDriverWait(driver, 3).until(EC.url_contains("/dashboard"))
    rows = dp.get_rows()
    assert any("Nguyen Van Kiem Thu (Da Cap Nhat)" in r.text for r in rows)

def test_tc06a_cancel_delete_student(driver, base_url):
    dp = DashboardPage(driver, base_url)
    dp.load()
    rows = dp.get_rows()
    target = None
    for r in rows:
        if "SVTEST01" in r.text:
            target = r
            break
    assert target is not None
    delete_btn = target.find_element(By.CSS_SELECTOR, "form button[type='submit'], .btn-delete")
    delete_btn.click()
    alert = driver.switch_to.alert
    alert.dismiss()
    rows_after = dp.get_rows()
    assert any("SVTEST01" in r.text for r in rows_after)

def test_tc06_delete_student(driver, base_url):
    dp = DashboardPage(driver, base_url)
    dp.load()
    rows = dp.get_rows()
    target = None
    for r in rows:
        if "SVTEST01" in r.text:
            target = r
            break

```

```

        assert target is not None
        delete_btn = target.find_element(By.CSS_SELECTOR, "form button[type='submit'],
.btn-delete")
        delete_btn.click()
        alert = driver.switch_to.alert
        alert.accept()
        WebDriverWait(driver, 3).until(EC.url_contains("/dashboard"))
        rows = dp.get_rows()
        assert not any("SVTEST01" in r.text for r in rows)

def test_tc07_add_student_invalid_email(driver, base_url):
    dp = DashboardPage(driver, base_url)
    dp.load()
    dp.click_add()
    WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.NAME, "name"))))
    driver.find_element(By.NAME, "name").send_keys("Sinh Vien Test Email")
    driver.find_element(By.NAME, "usn").send_keys("SV_EMAIL_BAD")
    driver.find_element(By.NAME, "branch").send_keys("Cong nghe thong tin")
    driver.find_element(By.NAME, "year").send_keys("2")
    email_field = driver.find_element(By.NAME, "email")
    email_field.send_keys("email_khong_hop_le_chua_domain")
    driver.execute_script("document.querySelector('form').noValidate = true;")
    driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()
    WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.CSS_SELECTOR, ".flash.danger"))))
    flash_text = driver.find_element(By.CSS_SELECTOR, ".flash.danger").text
    assert any(kw in flash_text for kw in ["email khong hop le", "Invalid email"])

def test_tc08_add_student_duplicate_usn(driver, base_url):
    dp = DashboardPage(driver, base_url)
    dp.load()
    dp.click_add()
    WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.NAME, "name"))))
    driver.find_element(By.NAME, "name").send_keys("Sinh Vien Trung Ma")
    driver.find_element(By.NAME, "usn").send_keys("SV22001")
    driver.find_element(By.NAME, "branch").send_keys("Tri tue nhan tao")
    driver.find_element(By.NAME, "year").send_keys("1")
    driver.find_element(By.NAME, "email").send_keys("trungma@donga.edu.vn")
    driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()
    WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.CSS_SELECTOR, ".flash.danger"))))
    flash_text = driver.find_element(By.CSS_SELECTOR, ".flash.danger").text
    assert any(kw in flash_text for kw in ["da ton tai", "already exists"])

def test_tc09_add_student_empty_required_fields(driver, base_url):
    dp = DashboardPage(driver, base_url)
    dp.load()
    dp.click_add()
    WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.NAME, "name"))))

```

```
driver.find_element(By.NAME, "branch").send_keys("Cong nghe thong tin")
driver.find_element(By.NAME, "year").send_keys("2")
driver.execute_script("document.querySelector('form').noValidate = true;")
driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()
WebDriverWait(driver, 3).until(EC.visibility_of_element_located((By.CSS_SELECTOR, ".flash.danger")))
flash_text = driver.find_element(By.CSS_SELECTOR, ".flash.danger").text
assert any(kw in flash_text for kw in ["day du", "required", "bat buoc"])
```

DANH MỤC TÀI LIỆU THAM KHẢO

Sách:

1. Hoàng Kiếm, Đỗ Văn Nhơn (2012). *Giáo trình Kiểm thử phần mềm*. Hà Nội: Nhà xuất bản Đại học Quốc gia Hà Nội.
2. Đinh Văn Đức (2001). *Ngữ pháp tiếng Việt – Từ loại*. Hà Nội: Nhà xuất bản Đại học Quốc gia Hà Nội.
3. Nguyễn Thiện Giáp (2008). *Giáo trình Ngôn ngữ học*. Hà Nội: Nhà xuất bản Đại học Quốc gia Hà Nội.
4. Calfee, R. C., & Valencia, R. R. (1991). *APA guide to preparing manuscripts for journal publication*. Washington, DC: American Psychological Association.
5. Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3rd ed.). Hoboken, NJ: John Wiley & Sons.
6. Okken, B. (2022). *Python Testing with pytest: Simple, Rapid, Effective, and Scalable* (2nd ed.). Raleigh, NC: Pragmatic Bookshelf.
7. Burns, D. (2018). *Selenium WebDriver 3 Practical Guide*. Birmingham, UK: Packt Publishing.

Tài liệu truy cập trên mạng:

8. International Software Testing Qualifications Board (ISTQB) (2023). *Certified Tester Foundation Level (CTFL) Syllabus v4.0*, truy cập ngày 15/09/2026 từ trang web: <https://www.istqb.org/>.
9. The Selenium Project (2026). *Selenium WebDriver Documentation & Best Practices*, truy cập ngày 20/09/2026 từ trang web: <https://www.selenium.dev/documentation/>.
10. Pallets Projects (2026). *Flask Documentation - Testing Flask Applications*, truy cập ngày 22/09/2026 từ trang web: <https://flask.palletsprojects.com/>.
11. PyTest Development Team (2026). *pytest Documentation*, truy cập ngày 25/09/2026 từ trang web: <https://docs.pytest.org/>.

12. Chandu3024 (2024). *Student Management System (Flask) with Selenium Testing*, GitHub repository: <https://github.com/Chandu3024/student-management-system-flask>.